Software Re-Engineering

GraphQL hands on

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

You are tasked with developing a backend for an **Event Management System** using GraphQL. The system should allow users to:

- ✅ Create and manage events (title, date, location)
- ✅ Register attendees for events
- ✅ Issue tickets to attendees
- ✅ Query information about events and attendees



Define the Schema

Event
- id (Int)
- title (String)
- date (String)
- location (String)
- attendees (List of Attendee)

Attendee
- id (Int)
- name (String)
- email (String)
- tickets (List of Ticket)

Ticket
- id (Int)
- event (Event)
- price (Float)



```
type Event {
  id: ID!
  title: String!
  date: String!
  location: String!
  attendees: [Attendee]
}
```

```
type Attendee {
  id: ID!
  name: String!
  email: String!
  tickets: [Ticket]
}
```

```
type Ticket {
  id: ID!
  event: Event!
  price: Float!
}
```



```
type Query {
  events: [Event]
  event(id: ID!): Event
  attendees(eventId: ID!): [Attendee]
}

type Mutation {
 addAttendee(name: String!, email: String!, eventId: ID!, ticket: String!): Attendee
  createEvent(title: String!, date: String!, location: String!): Event
  registerAttendee(eventId: ID!, name: String!, email: String!): Attendee
  issueTicket(eventId: ID!, price: Float!): Ticket
}
```

Create the Resolvers

Implement resolvers for:

✅ **Query:**

- Get all events
- Get attendees by event
- Get ticket details

✅ **Mutation:**

- Create a new event
- Register an attendee for an event
- Issue a ticket



```
const events = [];
const attendees = [];
const tickets = [];
```



Fetch all events and link related data using parent resolvers

```
const resolvers = {
  Query: {
    events: () => events,
    event: (parent, args) => events.find(event => event.id === args.id),
    attendees: (parent, args) => attendees.filter(att => att.eventId ===
args.eventId),
  }
};
```



events: () => events

This resolver handles the events query:

- events is an **array** (likely defined elsewhere in the code) containing event objects.
- () => events is a **direct return** of the array — no processing or filtering is done.

Example events data

```
const events = [
  { id: '1', title: 'GraphQL Bootcamp', date: '2025-06-10', location: 'Online' },
  { id: '2', title: 'Apollo Conference', date: '2025-07-15', location: 'San Francisco' }
];
```

Example query

```
query {
  events {
    id
    title
    date
    location
  }
}
```

event: (parent, args) => events.find(event => event.id === args.id)

This resolver handles the event(id: ID!) query:

- (parent, args)
  - parent → The return value from the parent field (if any).
  - args → The arguments passed to the query (in this case, id).
- events.find(event => event.id === args.id)
  - find() searches the events array for an event where the id matches args.id. The find() method is used to find the first element in an array that satisfies a given condition.
  - Returns the first matching event or undefined if no match is found.

- **(att => att.eventId === args.eventId)**

This is an **arrow function** used as a **callback** inside the filter() method.

1.att – This is a **parameter** representing each element in the attendees array.

- When you call filter(), it loops through each element in the array.
- att represents the **current element** being processed.

2.=> – This is the arrow function syntax, which is shorthand for defining functions.

3.att.eventId === args.eventId – This is the condition:

- If att.eventId (the eventId property of the current attendee) matches args.eventId (the argument passed from the GraphQL query), the element is included in the result.

Example query

```
query {
  event(id: "1") {
    title
    date
    location
  }
}
```

- args.id = "1"
- events.find() searches for id = "1" → returns the matching event.

attendees: (parent, args) => attendees.filter(att => att.eventId === args.eventId)

This resolver handles the attendees(eventId: ID!) query:

- (parent, args)
  - parent → The parent field value (not used here).
  - args.eventId → The event ID passed in the query.
- attendees.filter(att => att.eventId === args.eventId)
  - filter() returns an array of attendees where eventId matches args.eventId.
  - The filter() method is used to return all elements in an array that satisfy a given condition.

Method	Purpose	Returns	Stops After Finding First Match?
`find()`	Finds a single element	First matching element (or `undefined`)	✅ Yes
`filter()`	Finds multiple elements	Array of all matching elements	❌ No

NOTE

GraphQL vs JavaScript in Resolvers

1. **GraphQL** is a **query language** — it defines the structure of the data and the operations (queries, mutations, subscriptions).
2. **Resolvers** are **server-side functions** written in a programming language (like **JavaScript**) to define how to **fetch or process the data**.
3. Resolvers are NOT written in GraphQL — they are implemented using a backend language like **JavaScript**, **Python**, **Go**, etc.



Create a new event:

```
const resolvers = {
  Mutation: {
    createEvent: (parent, args) => {
      const newEvent = { id: events.length + 1, ...args };
      events.push(newEvent);
      return newEvent;
    }
  }
};
```



const newEvent = { id: events.length + 1, ...args };

const newEvent = { ... }

- const → Declares a **constant** variable (value can't be reassigned).
- { ... } → Creates a **new object** using object literal syntax.

id: events.length + 1

- events.length → Returns the number of elements in the events array.
- events.length + 1 → Creates a **unique ID** by adding 1 to the length of the array (so that IDs are sequential).

...args → Spread Operator

- The ... (spread operator) takes all key-value pairs from the args object and **spreads** them into the new object.
- This is a quick way to add multiple properties from args into the new object without manually listing them.

If args = { title: 'GraphQL Bootcamp', date: '2025-06-10' },

{ ...args }

{ title: 'GraphQL Bootcamp', date: '2025-06-10' }

Register an attendee and link to the event:

```
registerAttendee: (parent, { eventId, name, email }) => {
  const newAttendee = { id: attendees.length + 1, name,
email, eventide, ticket };
  attendees.push(newAttendee);
  return newAttendee;
}
```

Issue a ticket and calculate revenue dynamically:

```
issueTicket: (parent, { eventId, price }) => {
  const newTicket = { id: tickets.length + 1, eventId, price };
  tickets.push(newTicket);
  return newTicket;
}
```



Calculate total revenue using resolver logic:

```
Event: {
  attendees: (parent) => attendees.filter(att => att.eventId === parent.id),
  totalRevenue: (parent) => {
    return tickets
      .filter(ticket => ticket.eventId === parent.id)
      .reduce((total, ticket) => total + ticket.price, 0);
  }
}
```

The reduce() method reduces an array to a **single value** by executing a function on each element.

Syntax:

array.reduce(callback, initialValue)

Parameters:

- callback → A function that takes four arguments:
  - accumulator → The accumulated result of the function.
  - currentValue → The current element being processed.
  - index (optional) → The index of the current element.
  - array (optional) → The array being processed.
- initialValue → The starting value for the accumulator (optional).

The result of each callback becomes the new accumulator value.1

```
const numbers = [1, 2, 3, 4, 5];
const sum = numbers.reduce((acc, num) => acc + num, 0);


console.log(sum); // Output: 15
```

Process:

1.acc = 0, num = 1 → 0 + 1 = 1
2.acc = 1, num = 2 → 1 + 2 = 3
3.acc = 3, num = 3 → 3 + 3 = 6
4.acc = 6, num = 4 → 6 + 4 = 10
5.acc = 10, num = 5 → 10 + 5 = 15

Final result: 15

Library Management System

You are tasked with developing a library management system where:

- The library contains multiple **books** and **authors**.
- Each book belongs to one author, but an author can have multiple books.
- The system should allow adding new books and authors and retrieving information about books and authors.

You need to create the following GraphQL schema:

1. Book Type

•id (ID) – Unique identifier for the book
•title (String) – Title of the book
•authorId (ID) – ID of the author who wrote the book
•publishedYear (Int) – Year the book was published

2. Author Type

•id (ID) – Unique identifier for the author
•name (String) – Name of the author
•books ([Book]) – List of books written by the author

3. Query

- books – Return a list of all books
- book(id: ID!) – Return a single book by ID
- authors – Return a list of all authors
- author(id: ID!) – Return a single author by ID

4. Mutation

- addBook(title: String!, authorId: ID!, publishedYear: Int!) – Add a new book
- addAuthor(name: String!) – Add a new author

Task 1: Create GraphQL Schema

Write a GraphQL schema using the structure described above.



Solution

```
type Book {
  id: ID!
  title: String!
  author: Author!
  publishedYear: Int!
}

type Author {
  id: ID!
  name: String!
  books: [Book!]!
}

type Query {
  books: [Book!]!
  book(id: ID!): Book
  authors: [Author!]!
  author(id: ID!): Author
}
```

```
type Mutation {
  addBook(title: String!, authorId: ID!, publishedYear:
Int!): Book!
  addAuthor(name: String!): Author!
}
```



Task 2: Write Resolvers

Create resolvers to:

1. Fetch all books
2. Fetch a single book by ID
3. Fetch all authors
4. Fetch a single author by ID
5. Add a new book
6. Add a new author



```
const resolvers = {
  Query: {
    books: () => books,
    book: (parent, { id }) => books.find(book => book.id === Number(id)),
    authors: () => authors,
    author: (parent, { id }) => authors.find(author => author.id === Number(id)),
  },

  Mutation: {
    addBook: (parent, { title, authorId, publishedYear }) => {
      const authorExists = authors.some(author => author.id === Number(authorId));
      if (!authorExists) {
        throw new Error(`Author with ID ${authorId} does not exist.`);
      }
```



```
    const newBook = {
      id: books.length + 1,
      title,
      authorId: Number(authorId),
      publishedYear
    };
    books.push(newBook);
    return newBook;
  },

  addAuthor: (parent, { name }) => {
    const newAuthor = { id: authors.length + 1, name };
    authors.push(newAuthor);
    return newAuthor;
  }
},
```



```
  Book: {
    author: (book) => authors.find(author => author.id === book.authorId)
  },

  Author: {
    books: (author) => books.filter(book => book.authorId === author.id)
  }
};
```



Task 3: Write Queries and Mutations

Write the following GraphQL operations:

1. **Query** to get all books

```
query {
  books {
    id
    title
    author {
      name
    }
    publishedYear
  }
}
```

Task 3: Write Queries and Mutations

Write the following GraphQL operations:

2. Query to get a specific book by ID

```
query {
  book(id: 1) {
    title
    author {
      name
    }
    publishedYear
  }
}
```

Task 3: Write Queries and Mutations

Write the following GraphQL operations:

3. Query to get all authors

```
query {
  authors {
    id
    name
    books {
      title
      publishedYear
    }
  }
}
```

Task 3: Write Queries and Mutations

Write the following GraphQL operations:

Mutation to add a new book

RESOLVER

```
mutation {
  addBook(title: "Harry Potter", authorId: 3, publishedYear: 1997) {
    id
    title
    author {
      name
    }
    publishedYear
  }
}
```

```
type Mutation {
  addBook(title: String!, authorId: ID!, publishedYear: Int!):
Book!
}
```

Task 3: Write Queries and Mutations

Write the following GraphQL operations:

Mutation to add a new author

RESOLVER

```
Mutation: {
  addAuthor: (parent, { name }) => {
    const newAuthor = {
      id: authors.length + 1,
      name
    };
    authors.push(newAuthor);
    return newAuthor;
  }
}
```

```
mutation {
  addAuthor(name: "J.K. Rowling") {
    id
    name
  }
}
```

Task 4: Handle Nested Data

When querying an **author**, make sure to return a list of books written by that author using resolvers.

nested resolver within the Author type to map the author's books

```
const resolvers = {
  Query: {
    authors: () => authors,
    books: () => books,
  },

  Author: {
    books: (parent) => {
      return books.filter(book => book.authorId ===
parent.id);
    }
  }
};
```

